Code-Layout, Readability and Reusability

Date	19 September 2026
Team ID	SWTID-2026-2630
Project Name	AI Fit Track API
Maximum Marks	2 Marks

Code Layout, Readability and Reusability

AI Fit Track follows a modular MVC-inspired layout under `src/`, keeping every concern in its own folder so a new route can be added without touching unrelated files.

Folder	Purpose	Reusability Practice
`config/`	MongoDB connection setup (`db.js`)	Connection logic centralized once; every model imports the same connected instance
`controllers/`	Request-handling logic per resource (auth, workout, AI)	Thin controllers delegate persistence to Models and AI calls to `services/`, so logic isn't duplicated
`middleware/`	JWT auth guard, centralized error handler	A single `auth.js` is reused across every protected route instead of re-checking tokens per controller
`models/`	Mongoose schemas (`User.js` , `Workout.js`) with validation baked in	Validation rules (required, enum, min) live once in the schema, not repeated in every controller
`routes/`	Endpoint-to-controller mapping per resource	Each route file is mounted with its own prefix (`/api/auth` , `/api/workouts` , `/api/ai`), keeping URL structure predictable
`services/`	`geminiService.js` – isolates all Google Gemini prompt/response handling	Both AI endpoints (recommendation, insights) call the same service module with different prompt builders, avoiding duplicated Gemini client setup

Consistent naming (camelCase functions, PascalCase models), small single-purpose functions per controller action, and comments above non-obvious logic (e.g., the Gemini prompt templates) keep the codebase readable for all 5 contributors working in parallel.